

AI Coding Interviewer (MVP) – Full Project Specification

Project Overview

Build a complete, production-quality full-stack web application called _____ (working title).

The application is an AI-powered mock technical interview platform that allows users to practice coding interviews in a realistic environment.

Unlike ChatGPT, the AI should behave like an actual software engineering interviewer. It should ask the candidate about their thought process, challenge assumptions, ask follow-up questions, provide hints only when appropriate, and generate detailed interview feedback at the end.

The application should feel like a polished startup product rather than a school assignment.

The primary objective is **user experience and realism**, not implementing every advanced feature.

This MVP intentionally keeps the scope small so the application can be completed and polished before adding more advanced functionality.

Project Goals

The finished application should allow a user to:

- Select a coding interview question.
- Read the problem statement.
- Write Python code inside a professional code editor.
- Have an AI interviewer conduct a realistic interview through text chat.
- Run their solution against test cases.
- Finish the interview.
- Receive detailed AI-generated feedback.

The project should emphasize clean UI, strong architecture, modular code, and maintainability.

Out of Scope (Do NOT Build)

The following features are intentionally excluded from Version 1.

- Voice conversations
- Video/Webcam
- Authentication
- User accounts
- Cloud deployment
- Docker sandboxing
- Multiple programming languages
- Multiplayer interviews
- System Design interviews
- Behavioral interviews
- Resume uploads
- Interview history
- Premium subscriptions
- Payment processing
- Admin dashboards
- Notifications
- Email
- OAuth

These can all be added later.

Tech Stack

Frontend

- Next.js 15
 - React
 - TypeScript
 - TailwindCSS
 - shadcn/ui
 - Monaco Editor
-

Backend

- FastAPI
 - Python
-

AI

Initially support:

- Ollama

The architecture should make it easy to replace Ollama with OpenAI or another provider later.

Database

Use SQLite.

SQLite is sufficient for this project.

No ORM is required if direct SQL is simpler, although SQLAlchemy is acceptable.

Application Pages

1. Landing Page

Create a modern SaaS-style landing page.

Sections:

Hero

Feature cards

How it Works

Call-to-action button

Footer

The design should look clean and professional.

2. Problem Selection Page

Display coding problems as cards.

Each card contains:

Problem title

Difficulty

Topics

Estimated interview time

Start Interview button

Initially populate the database with approximately **200–300 Python interview problems** covering common topics such as:

Arrays

Strings

Hash Maps

Two Pointers

Sliding Window

Stacks

Queues

Linked Lists

Binary Trees

BSTs

Graphs

DFS

BFS

Heaps

Backtracking

Dynamic Programming

Intervals

Binary Search

Greedy

The problems do **not** need to be original. They can mirror classic interview problems, but avoid copying proprietary wording verbatim. The goal is to provide a representative practice set.

3. Interview Page

The interview page is the heart of the application.

Use a responsive three-column layout.

Left Panel

Displays:

Problem title

Difficulty

Description

Input format

Output format

Examples

Constraints

Hints (collapsed)

Center Panel

Monaco Editor

Python syntax highlighting

Starter code

Run button

Reset button

Output console

Execution results

Right Panel

AI Interview Chat

Conversation history

Typing indicator

Chat input

Send button

Finish Interview button

AI Interview Behavior

The AI must act like an experienced software engineering interviewer.

It should never behave like a generic chatbot.

The AI should:

Introduce the interview.

Ask the candidate to explain their approach before coding.

Ask about runtime complexity.

Ask about space complexity.

Ask about tradeoffs.

Ask about edge cases.

Ask clarification questions.

Encourage the candidate to think.

Provide progressively larger hints only when requested.

Never immediately provide the entire solution.

Never paste the completed code.

The AI should maintain interview context throughout the conversation.

Example Interview Flow

Welcome

↓

Candidate introduction

↓

Problem presented

↓

Candidate explains approach

↓

AI asks follow-up questions

↓

Candidate writes code

↓

Candidate runs tests



AI comments on failures if appropriate



Candidate improves solution



Candidate finishes interview



AI generates report

Code Editor

Use Monaco Editor.

Requirements:

Dark theme

Line numbers

Syntax highlighting

Auto indentation

Copy support

Resizable

Professional appearance

Only Python is supported.

Code Execution

Users should be able to execute Python code.

Execution should return:

stdout

stderr

runtime exceptions

execution time

pass/fail status

Visible test case results

Execution results should appear directly below the editor.

Problem Database

SQLite database.

Each problem should contain:

ID

Title

Difficulty

Topics

Description

Input description

Output description

Constraints

Example inputs

Example outputs

Starter code

Visible test cases

Hidden test cases

Expected outputs

The database should contain approximately **200–300 problems**.

AI Context

Each chat request sent to the LLM should include:

Current problem

Conversation history

Current Python code

Run results

Interview stage

Hints already provided

The AI should always understand the current interview state.

Finish Interview

When the user presses Finish Interview:

Send:

Entire conversation

Current code

Run history

Problem information

to the LLM.

Generate a structured interview report.

Interview Report

Display a dedicated report page.

Include:

Overall score

Communication

Problem solving

Code quality

Optimization

Edge case handling

Strengths

Weaknesses

Suggestions

Overall interview summary

The report should look modern and visually appealing.

UI Design

Design should resemble a modern SaaS product.

Requirements:

Dark mode

Rounded cards

Smooth transitions

Responsive layout

Professional typography

Consistent spacing

Minimalist design

No placeholder styling

No unfinished pages

Every page should feel polished.

Components

The frontend should use reusable components.

Examples:

Navbar

Sidebar

Problem Card

Difficulty Badge

Code Editor

Console

Chat Bubble

Typing Indicator

Loading Spinner

Toast Notifications

Feedback Card

Score Card

Section Header

Buttons

Dialogs

Backend API

Create REST endpoints.

GET

/problems

Returns all interview problems.

GET

/problems/{id}

Returns one problem.

POST

/chat

Processes AI interviewer messages.

POST

/run

Executes Python code.

POST

/finish

Generates final interview report.

Coding Standards

Use:

Clean architecture

Reusable components

TypeScript best practices

Modular backend

Meaningful naming

Small files

No duplicated logic

Document important functions

Avoid unnecessary complexity

Future Roadmap (Do Not Implement)

Future versions may include:

Voice conversations

Speech recognition

Text-to-speech

Behavioral interviews

System design interviews

Company-specific interview modes

Resume-based interviews

Docker sandbox

Multiple languages

Authentication

Cloud deployment

Analytics dashboard

Interview history

User profiles

Leaderboards

Success Criteria

The MVP is considered complete when a user can:

1. Open the website.
2. Choose a coding problem from a library of approximately 200–300 problems.
3. Read the full problem description.
4. Write Python code in a Monaco editor.
5. Execute their code against test cases and view results.
6. Hold a realistic text-based conversation with an AI interviewer that asks thoughtful questions and provides hints without revealing the full solution.
7. Finish the interview.
8. Receive a polished AI-generated feedback report with scores, strengths, weaknesses, and actionable suggestions.

The application should be stable, responsive, visually polished, and structured so that advanced features like voice, authentication, and additional programming languages can be added in future versions without major architectural changes.

root/

frontend/

app/

components/

hooks/

lib/

styles/

backend/

app/

api/

services/

models/

database/

prompts/

utils/

database/

interview.db

shared/

docs/

README.md